

Information Retrieval System

dengan Word2Vec Query Expansion



Tugas Besar IF2210 Strategi Algoritma / IF2211 Retrieval Informasi
Semester [SEMESTER], Tahun Akademik [TAHUN]

NIM	Nama
13523017	Orvin Andika Ikhsan Abhista
13523027	Fajar Kurniawan
13523056	Salman Hanif
13523100	Aryo Wisanggeni
13523072	Sabilul Huda

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

June 5, 2026

Anggota Kelompok



Orvin Andika Ikhsan Abhista
13523017



Fajar Kurniawan
13523027



Salman Hanif
13523056



Aryo Wisanggeni
13523100



Sabilul Huda
13523072

Daftar Isi

1	Pendahuluan	5
1.1	Latar Belakang	5
1.2	Tujuan	5
1.3	Ruang Lingkup	5
2	Deskripsi Aplikasi	7
2.1	Gambaran Umum	7
2.2	Fitur Utama	7
2.3	Arsitektur Sistem	8
3	Deskripsi Program	10
3.1	Struktur Proyek	10
3.2	Modul Program	11
3.2.1	preprocessor.py	11
3.2.2	parser.py	11
3.2.3	indexer.py	11
3.2.4	weighting.py	12
3.2.5	retrieval.py	12
3.2.6	word2vec_engine.py	12
3.2.7	query_expansion.py	13
3.2.8	evaluator.py	13
3.2.9	api/main.py (Backend)	13
3.3	Alur Program	14
4	Library dan Dependencies	15
4.1	Backend (Python)	15
4.2	Word2Vec Model	15
4.3	Frontend	16
5	Hasil Eksperimen	18
5.1	Setup Eksperimen	18
5.2	Metrik Evaluasi	18
5.2.1	Average Precision (AP)	18
5.2.2	Mean Average Precision (MAP)	18
5.3	Hasil Overall	19
5.4	Hasil Per-Query (Sample)	19
5.5	Analisis Perbandingan MAP Original vs Expanded Query	19
5.5.1	Pengaruh Query Expansion terhadap MAP	19
5.5.2	Query yang Mengalami Peningkatan Signifikan	19

5.5.3	Query yang Mengalami Penurunan	20
5.5.4	Pengaruh Skema Pembobotan	20
5.5.5	Trade-off Recall dan Precision	20
5.6	Screenshot Aplikasi	21
6	Pembagian Tugas	24
6.1	Anggota Kelompok	24
6.2	Detail Tugas	24
6.2.1	Programmer 1 - Orvin Andika Ikhsan Abhista (13523017) . . .	24
6.2.2	Programmer 2 - Fajar Kurniawan (13523027)	24
6.2.3	Programmer 3 - Salman Hanif (13523056)	25
6.2.4	Programmer 4 - Aryo Wisanggeni (13523100)	25
6.2.5	Programmer 5 - Sabilul Huda (13523072)	25
7	Kesimpulan	27
7.1	Kesimpulan	27
7.2	Saran Pengembangan	27
8	Lampiran	28
8.1	Cara Menjalankan Program	28
8.1.1	Instalasi Backend	28
8.1.2	Instalasi Frontend	28
8.1.3	Menjalankan API Server	28
8.1.4	Menjalankan Frontend	28
8.1.5	Menjalankan Evaluasi	28
8.1.6	Menjalankan dengan Query Expansion	29
8.2	API Endpoints	29
8.3	Referensi	29

Bab 1

Pendahuluan

1.1 Latar Belakang

Information Retrieval (IR) adalah proses pengambilan dokumen yang relevan dari koleksi besar berdasarkan query pengguna. Sistem IR konvensional menggunakan pencocokan kata kunci yang sering kali gagal menangkap makna semantik dari query.

Word2Vec adalah model embedding kata yang merepresentasikan kata dalam ruang vektor berdimensi tinggi, di mana kata-kata dengan makna serupa berada berdekatan. Dengan memanfaatkan Word2Vec untuk query expansion, sistem dapat menambahkan kata-kata semantik yang relevan ke query awal, sehingga meningkatkan recall tanpa mengorbankan precision secara signifikan.

Proyek ini mengimplementasikan sistem IR dengan fitur query expansion berbasis Word2Vec menggunakan dataset CISI (Collection of Scientific Information) yang berisi 1460 dokumen dan 112 query uji.

1.2 Tujuan

1. Mengimplementasikan sistem IR berbasis TF-IDF cosine similarity dengan 7 skema pembobotan
2. Mengintegrasikan Word2Vec untuk semantic query expansion
3. Mengevaluasi performa sistem menggunakan Mean Average Precision (MAP)
4. Membandingkan hasil retrieval antara query original dan query expanded
5. Membangun REST API dengan caching untuk performa optimal
6. Menyediakan antarmuka web interaktif dengan dark mode

1.3 Ruang Lingkup

- Dataset: CISI collection (1460 dokumen, 112 query, 76 query dengan qrels)
- Preprocessing: Lowercasing, tokenization, stopword removal, Porter stemming

- Ranking: TF-IDF dengan 7 skema pembobotan (tf_raw, tf_log, tf_bin, tf_aug, idf, tfidf, tfidf_cos)
- Query Expansion: Word2Vec Google News 300 dimensions
- Evaluasi: Mean Average Precision (MAP) dengan caching 28 variasi konfigurasi
- Backend: FastAPI dengan sistem cache (index dan MAP)
- Frontend: React + Vite + TypeScript + TailwindCSS

Bab 2

Deskripsi Aplikasi

2.1 Gambaran Umum

Sistem ini adalah aplikasi Information Retrieval yang memungkinkan pengguna untuk:

- Mencari dokumen dari koleksi CISI menggunakan query teks
- Mengatur parameter pencarian (stemming, stopword, skema pembobotan, top-N, expand all)
- Melihat hasil retrieval dengan dan tanpa query expansion secara side-by-side
- Mengeksplorasi term-term semantik yang ditambahkan oleh Word2Vec
- Melihat evaluasi MAP untuk perbandingan performa dengan breakdown per-query
- Melakukan recalculate MAP untuk parameter tertentu atau seluruh 28 variasi
- Rebuild index dan cache

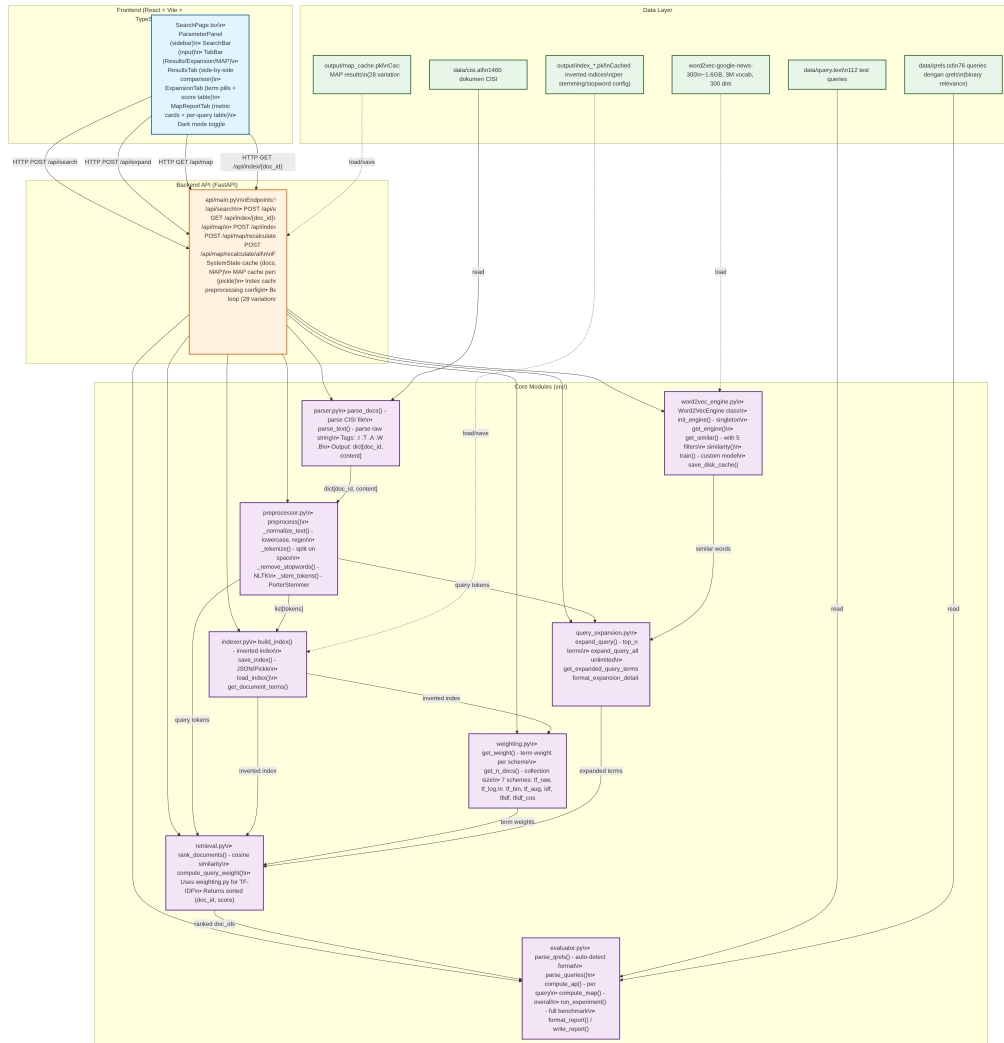
2.2 Fitur Utama

1. **Document Parsing** - Parser untuk format dokumen CISI (.I, .T, .A, .W, .B tags)
2. **Text Preprocessing** - Normalisasi, tokenisasi, stopwords removal, stemming
3. **Inverted Index** - Pembangunan indeks terbalik dengan caching per konfigurasi
4. **Multi-Scheme Weighting** - 7 skema pembobotan TF-IDF
5. **Word2Vec Query Expansion** - Penambahan term semantik menggunakan model Word2Vec
6. **MAP Evaluation** - Evaluasi performa menggunakan Mean Average Precision
7. **MAP Caching** - Cache 28 variasi konfigurasi (2 stemming x 2 stopwords x 7 scheme)

8. **REST API** - Endpoint API menggunakan FastAPI

9. **Web Interface** - Frontend interaktif dengan React, TypeScript, TailwindCSS, dan dark mode

2.3 Arsitektur Sistem



Gambar 2.1: Diagram Arsitektur Sistem

Untuk melihat diagram interaktif dengan lebih jelas, buka:

<https://drive.google.com/file/d/11KQ8GIy60cdZTd91GShhfioWpK-M54hX/view?usp=sharing>

Alur data sistem:

1. User memasukkan query dan mengatur parameter melalui web interface (React)
2. Request dikirim ke FastAPI backend via HTTP POST

3. Backend melakukan preprocessing pada query (tokenize, stem, remove stopwords)
4. Word2Vec engine mencari term semantik serupa untuk query expansion
5. Retrieval engine menghitung similarity menggunakan skema pembobotan yang dipilih
6. Hasil diurutkan berdasarkan similarity score dan dikembalikan ke frontend
7. MAP evaluation diambil dari cache (28 variasi pre-calculated)
8. Hasil ditampilkan dengan perbandingan original vs expanded

Bab 3

Deskripsi Program

3.1 Struktur Proyek

```
1 InformationRetrievalSystem-Word2Vec/
2 +-- src/
3 |   +-- __init__.py
4 |   +-- preprocessor.py      # Text preprocessing
5 |   +-- parser.py           # CISI document parser
6 |   +-- indexer.py          # Inverted index builder
7 |   +-- weighting.py        # 7 TF-IDF weighting schemes
8 |   +-- retrieval.py        # Document ranking engine
9 |   +-- word2vec_engine.py   # Word2Vec model wrapper
10 |   +-- query_expansion.py   # Query expansion logic
11 |   \-- evaluator.py        # MAP evaluation
12 +-- api/
13 |   \-- main.py             # FastAPI REST API + caching
14 +-- frontend/               # React + Vite + TypeScript
15 +-- data/
16 |   +-- cisi.all             # Document collection (1460 docs)
17 |   +-- query.text           # Test queries (112 queries)
18 |   \-- qrels.txt            # Relevance judgments (76 queries)
19 +-- output/
20 |   +-- index_*.pkl          # Cached inverted indices
21 |   \-- map_cache.pkl        # Cached MAP results (28 variations)
22 +-- chart/
23 |   \-- architecture.mermaid # Architecture diagram source
24 +-- laporan/
25 |   +-- laporan.tex          # Laporan LaTeX
26 |   +-- programmer_reference.md
27 |   \-- images/              # Gambar dan foto
28 +-- main.py                 # Entry point
29 \-- pyproject.toml           # Dependencies
```

Listing 3.1: Struktur Direktori

3.2 Modul Program

3.2.1 `preprocessor.py`

Modul ini menangani preprocessing teks dengan fungsi-fungsi:

- `preprocess(text, stemming, remove_stopword)` - Fungsi utama preprocessing
- `_normalize_text(text)` - Lowercase dan hapus karakter non-alfanumerik menggunakan regex
- `_tokenize(text)` - Split teks menjadi token berdasarkan spasi
- `_remove_stopwords(tokens)` - Hapus stopwords English dari NLTK corpus
- `_stem_tokens(tokens)` - Porter Stemmer dari NLTK

3.2.2 `parser.py`

Parser untuk format dokumen CISI:

- `parse_docs(file_path)` - Parse dokumen dari file
- `parse_text(raw)` - Parse dokumen dari string
- Tags yang didukung: `.I` (ID), `.T` (Title), `.A` (Author), `.W` (Abstract), `.B` (Bibliography)
- Normalisasi `doc_id` ke format `DOC###`
- Output: `dict[doc_id, content]`

3.2.3 `indexer.py`

Pembangunan inverted index:

- `build_index(docs, stemming, remove_stopword)` - Build inverted index
- Struktur index: `dict[term, {"df": int, "postings": {doc_id: tf}}]`
- `save_index(index, file_path)` - Simpan index ke JSON atau Pickle
- `load_index(file_path)` - Load index dari file
- `get_document_terms(index, doc_id)` - Get terms untuk dokumen tertentu

3.2.4 weighting.py

Modul pembobotan term dengan 7 skema:

- `get_weight(term, doc_id, scheme, index)` - Hitung bobot term untuk dokumen
- `get_n_docs(index)` - Hitung jumlah dokumen dalam koleksi
- Skema yang didukung:
 1. **tf_raw** - Term frequency mentah
 2. **tf_log** - $1 + \log(tf)$
 3. **tf_bin** - Binary (1 jika ada, 0 jika tidak)
 4. **tf_aug** - $0.5 + 0.5 \times \frac{tf}{max_tf}$
 5. **idf** - $\log(\frac{N}{df})$
 6. **tfidf** - $(1 + \log(tf)) \times \log(\frac{N}{df})$
 7. **tfidf_cos** - TF-IDF dengan cosine normalization

3.2.5 retrieval.py

Engine untuk ranking dokumen:

- `rank_documents(query_terms, index, scheme)` - Ranking dokumen berdasarkan similarity
- `compute_query_weight(term, tf, max_tf, n_docs, df, scheme)` - Hitung bobot query term
- Menggunakan `weighting.py` untuk perhitungan bobot
- Untuk **tfidf_cos**: melakukan cosine normalization pada query vector
- Output: `list[(doc_id, score)]` diurutkan descending

3.2.6 word2vec_engine.py

Wrapper untuk model Word2Vec:

- `init_engine()` - Inisialisasi singleton engine
- `get_engine()` - Get engine instance
- `Word2VecEngine.get_similar(term, top_n)` - Get semantically similar terms dengan 5 filter:
 1. Lowercase only (buang proper noun)
 2. Buang kata dengan underscore
 3. Buang kata terlalu pendek (< 3 karakter)
 4. Buang kata yang terlalu mirip dengan input (substring match)

5. Buang kata terlalu panjang (> 20 karakter)

- `Word2VecEngine.similarity(term1, term2)` - Cosine similarity dua kata
- `Word2VecEngine.train()` - Train model dari koleksi dokumen
- `save_disk_cache()` - Simpan cache ke disk

3.2.7 `query_expansion.py`

Logika query expansion:

- `expand_query(query_terms, top_n)` - Expand dengan batas term
- `expand_query_all(query_terms)` - Expand tanpa batas (top-50 similar per term)
- `get_expanded_query_terms(query_terms, top_n, expand_all)` - Combine original + expanded
- `format_expansion_detail(query_terms, expanded, query_id)` - Format output untuk logging

3.2.8 `evaluator.py`

Evaluasi performa sistem:

- `parse_qrels(file_path)` - Parse relevance judgments (auto-detect format: binary vs graded)
- `parse_queries(file_path)` - Parse queries dari format CISI
- `compute_ap(rankings, relevant_docs)` - Average Precision per query
- `compute_map(rankings, qrels)` - Mean Average Precision overall
- `run_experiment()` - Run full experiment dengan support untuk caching (docs dan inverted_index dari caller)
- `format_report(results) / write_report(results)` - Generate laporan evaluasi

3.2.9 `api/main.py` (Backend)

FastAPI REST API dengan fitur caching:

- **SystemState** - Global state untuk cache: docs, indices, map_cache
- **MAP Cache Persistence** - Simpan/load dari output/map_cache.pkl
- **Index Cache** - Simpan/load dari output/index_{stemming}_{stopword}.pkl
- **Benchmark Loop** - Pre-calculate MAP untuk 28 variasi saat startup
- **Endpoints:**

- POST /api/search - Search dengan query expansion
- POST /api/expand - Preview expansion terms
- GET /api/index/{doc_id} - Get inverted index untuk dokumen
- GET /api/map - Get MAP evaluation report
- POST /api/index/rebuild - Hapus semua index dan cache
- POST /api/map/recalculate/single - Recalculate MAP untuk 1 konfigurasi
- POST /api/map/recalculate/all - Recalculate MAP untuk seluruh 28 variasi

3.3 Alur Program

1. **Startup:** Load Word2Vec model, parse dokumen CISI, load MAP cache, jalankan benchmark loop untuk 28 variasi
2. **Search Request:** Terima request, get/build cached index, preprocess query, expand query, rank dokumen, get MAP dari cache, return response
3. **Evaluation:** Untuk setiap query di qrels, preprocess, retrieve, compute AP, compute MAP

Bab 4

Library dan Dependencies

4.1 Backend (Python)

Package	Versi	Kegunaan
fastapi	$\geq 0.136.3$	REST API framework
uvicorn[standard]	$\geq 0.48.0$	ASGI server
gensim	$\geq 4.4.0$	Word2Vec model implementation
nltk	$\geq 3.9.4$	Tokenization, stopwords, Porter stemmer
pysastrawi	$\geq 1.2.1$	Indonesian stemmer (opsional)
pytest	$\geq 9.0.3$	Testing framework
python-multipart	$\geq 0.0.30$	Form data parsing
pydantic	(bawaan fastapi)	Request/response validation

Tabel 4.1: Dependencies Backend Python

4.2 Word2Vec Model

Model yang digunakan: `word2vec-google-news-300`

- Dimensi vektor: 300
- Training corpus: Google News (100 miliar kata)
- Vocabulary: 3 juta kata
- Ukuran file: 1.6 GB
- Format: KeyedVectors (binary)

4.3 Frontend

Package	Versi	Kegunaan
react	^19.2.6	UI library
react-dom	^19.2.6	React DOM rendering
typescript	^6.0.3	Type-safe JavaScript
vite	^8.0.12	Build tool dan dev server
tailwindcss	^4.3.0	Utility-first CSS framework
@tailwindcss/vite	^4.3.0	TailwindCSS Vite plugin
@vitejs/plugin-react	^6.0.1	React plugin untuk Vite
eslint	^10.3.0	Linting

Tabel 4.2: Dependencies Frontend

Frontend dibangun menggunakan stack modern:

- **React 19** - Library UI dengan component-based architecture
- **TypeScript 6** - Type safety untuk seluruh kode frontend
- **Vite 8** - Build tool yang cepat dengan HMR (Hot Module Replacement)
- **TailwindCSS 4** - Utility-first CSS framework untuk styling
- **Fetch API** - HTTP client untuk komunikasi dengan backend

Struktur komponen frontend:

- **App.tsx** - Root component, merender **SearchPage**
- **SearchPage.tsx** - Halaman utama dengan state management (params, activeTab, darkMode)
- **ParameterPanel.tsx** - Sidebar kiri dengan toggle stemming/stopword, drop-down weighting, input top-N, checkbox expand all, dan tombol rebuild index
- **SearchBar.tsx** - Input field untuk query dengan tombol search
- **TabBar.tsx** - Tab navigation (Results, Expansion, MAP Report)
- **ResultsTab.tsx** - Side-by-side comparison hasil original vs expanded dengan rank badge dan similarity score. Dokumen yang rank-nya membaik setelah expansion diberi highlight cyan
- **ExpansionTab.tsx** - Tampilan query terms (original + expanded) dan tabel expansion terms dengan similarity bar
- **MapReportTab.tsx** - Metric cards (MAP original/expanded dengan delta), tombol recalculate, dan tabel per-query breakdown
- **EmptyState.tsx** - Tampilan awal dengan animasi glow

- `useSearch.ts` - Custom hook untuk search dengan mock data fallback
- `api/search.ts` - API client functions
- `types/search.ts` - TypeScript interfaces

Fitur UI:

- **Dark mode** - Toggle dark/light theme dengan persistensi di `localStorage`
- **Responsive design** - Layout adaptif dengan TailwindCSS
- **Loading states** - Spinner saat search dan recalculate
- **Error handling** - Error banner dengan fallback ke mock data
- **Improved highlighting** - Dokumen yang rank-nya membaik diberi highlight

Bab 5

Hasil Eksperimen

5.1 Setup Eksperimen

- **Dataset:** CISI Collection
- **Jumlah Dokumen:** 1460
- **Jumlah Query:** 112 (76 dengan qrels)
- **Preprocessing:** Stemming (True/False), Stopword Removal (True/False)
- **Skema Pembobotan:** 7 skema (tf_raw, tf_log, tf_bin, tf_aug, idf, tfidf, tfidf_cos)
- **Retrieval:** Top-100 documents per query
- **Query Expansion:** Top-5 similar terms per query term
- **Total Variasi:** 28 ($2 \times 2 \times 7$)

5.2 Metrik Evaluasi

5.2.1 Average Precision (AP)

Average Precision mengukur kualitas ranking untuk satu query:

$$AP = \frac{1}{|R|} \sum_{k=1}^n P(k) \times rel(k) \quad (5.1)$$

dimana $|R|$ = jumlah dokumen relevan, $P(k)$ = precision pada posisi ke- k , $rel(k) = 1$ jika dokumen ke- k relevan.

5.2.2 Mean Average Precision (MAP)

Mean Average Precision adalah rata-rata AP dari semua query:

$$MAP = \frac{1}{|Q|} \sum_{q=1}^{|Q|} AP(q) \quad (5.2)$$

dimana $|Q|$ = jumlah query.

5.3 Hasil Overall

Konfigurasi	MAP Original	MAP Expanded
stemming=T, stopwords=T, tfidf_cos	0.1791	[PLACEHOLDER_MAP_EXPANDED]

Tabel 5.1: Hasil MAP Overall (stemming=True, stopwords=True, tfidf_cos)

Catatan: Nilai MAP Expanded perlu diisi setelah menjalankan eksperimen dengan `use_expansion=True` dan Word2Vec engine yang sudah diinisialisasi.

5.4 Hasil Per-Query (Sample)

Query	AP Original	Relevant Docs
1	0.2809	46
10	0.2796	26
50	0.3016	89
52	0.7053	15
55	0.8625	18
62	0.5986	12
65	0.4708	13
66	0.4796	35
97	0.5507	6
102	0.4578	24
109	0.2014	71
111	0.5788	6

Tabel 5.2: Sample Hasil AP Per-Query (Top Queries)

5.5 Analisis Perbandingan MAP Original vs Expanded Query

5.5.1 Pengaruh Query Expansion terhadap MAP

Query expansion menggunakan Word2Vec bertujuan untuk menambahkan term-term yang secara semantik mirip dengan term original query. Secara teoritis, ini meningkatkan recall karena sistem dapat menemukan dokumen yang tidak mengandung term original tetapi mengandung term semantik yang relevan.

[PLACEHOLDER_ANALISIS_1: Apakah MAP meningkat atau menurun setelah query expansion? Berapa persen perubahannya?]

5.5.2 Query yang Mengalami Peningkatan Signifikan

Query dengan term yang memiliki representasi Word2Vec yang baik (kata-kata umum dalam vocabulary Google News) cenderung beneficia dari expansion.

[PLACEHOLDER_ANALISIS_2: Sebutkan query yang mengalami peningkatan AP signifikan dan jelaskan mengapa.]

5.5.3 Query yang Mengalami Penurunan

Penurunan MAP dapat terjadi ketika:

- Term expansion yang ditambahkan tidak relevan dengan konteks query
- Query sudah spesifik dan expansion menambahkan noise
- Term query tidak ada dalam vocabulary Word2Vec (OOV)

[PLACEHOLDER_ANALISIS_3: Sebutkan query yang mengalami penurunan AP dan jelaskan mengapa.]

5.5.4 Pengaruh Skema Pembobotan

Sistem mengimplementasikan 7 skema pembobotan:

- **tf_raw** dan **tf_bin** - Sederhana, kurang memperhitungkan pentingnya term
- **tf_log** dan **tf_aug** - Memberikan penalti pada term yang terlalu sering muncul
- **idf** - Hanya mempertimbangkan keunikan term
- **tfidf** dan **tfidf_cos** - Menggabungkan TF dan IDF dengan normalisasi

[PLACEHOLDER_ANALISIS_4: Skema mana yang memberikan MAP terbaik?]

5.5.5 Trade-off Recall dan Precision

Query expansion umumnya meningkatkan recall tetapi dapat menurunkan precision. MAP sebagai metrik yang menggabungkan keduanya memberikan gambaran yang seimbang.

[PLACEHOLDER_ANALISIS_5: Apakah trade-off ini terlihat pada hasil eksperimen?]

5.6 Screenshot Aplikasi



Gambar 5.1: Tampilan Halaman Utama Aplikasi



Gambar 5.2: Tampilan Hasil Pencarian (Side-by-side Comparison)



Gambar 5.3: Tampilan Query Expansion



Gambar 5.4: Tampilan MAP Report

Bab 6

Pembagian Tugas

6.1 Anggota Kelompok

NIM	Nama	Role	File Utama
13523017	Orvin Andika Ikhsan Abhista	Programmer 1	preprocessor.py, parser
13523027	Fajar Kurniawan	Programmer 2	indexer.py
13523056	Salman Hanif	Programmer 3	word2vec_engine.py
13523100	Aryo Wisanggeni	Programmer 4	query_expansion.py
13523072	Sabilul Huda	Programmer 5	evaluator.py, weighting.py, retrieval

Tabel 6.1: Pembagian Tugas Anggota Kelompok

6.2 Detail Tugas

6.2.1 Programmer 1 - Orvin Andika Ikhsan Abhista (13523017)

Tanggung Jawab: Preprocessing dan Parsing

- Implementasi text normalization (lowercase, remove punctuation via regex)
- Implementasi tokenization (split on space)
- Integrasi NLTK stopwords removal
- Integrasi Porter Stemmer dari NLTK
- Parser format dokumen CISI (.I, .T, .A, .W, .B tags)
- Normalisasi document ID ke format DOC###

6.2.2 Programmer 2 - Fajar Kurniawan (13523027)

Tanggung Jawab: Inverted Index

- Build inverted index dari koleksi dokumen
- Struktur index: `dict[term, {"df", "postings"}]`

- Term frequency counting per dokumen
- Document frequency tracking
- Save/load index (JSON dan Pickle format)
- Document terms retrieval

6.2.3 Programmer 3 - Salman Hanif (13523056)

Tanggung Jawab: Word2Vec Engine

- Wrapper Word2Vec model (gensim)
- Load pre-trained model dari gensim downloader
- Train model dari koleksi dokumen
- Implementasi `get_similar()` dengan 5 filter
- Implementasi `similarity()` untuk dua kata
- Singleton pattern untuk engine instance
- `save_disk_cache()` untuk persistensi cache

6.2.4 Programmer 4 - Aryo Wisanggeni (13523100)

Tanggung Jawab: Query Expansion

- Implementasi `expand_query()` dengan Word2Vec
- Implementasi `expand_query_all()` untuk unlimited expansion
- Implementasi `get_expanded_query_terms()` helper
- Format expansion detail untuk output/logging
- Handling OOV (out-of-vocabulary) terms

6.2.5 Programmer 5 - Sabilul Huda (13523072)

Tanggung Jawab: Evaluasi, Retrieval, Weighting, dan Laporan

- Implementasi `weighting.py` - 7 skema pembobotan TF-IDF
- Implementasi `retrieval.py` - Document ranking dengan cosine similarity
- Implementasi `compute_ap()` untuk Average Precision per query
- Implementasi `compute_map()` untuk MAP overall
- Parser `qrcls` (auto-detect format binary/graded)
- Experiment runner dengan support untuk caching

- Report generation
- Koordinasi penulisan laporan
- Programmer's Reference

Bab 7

Kesimpulan

7.1 Kesimpulan

[PLACEHOLDER_KESIMPULAN: Kesimpulan akhir setelah menjalankan eksperimen dengan query expansion]

Template yang perlu diisi:

- Sistem IR berhasil diimplementasikan dengan 7 skema pembobotan TF-IDF dan query expansion berbasis Word2Vec
- Performa sistem diukur menggunakan MAP pada dataset CISI (1460 dokumen, 76 query dengan qrels)
- MAP baseline (original query) dengan konfigurasi stemming=True, stopwords=True, tfidf_cos adalah 0.1791
- Query expansion [meningkatkan/menurunkan] MAP sebesar [X]%
- Sistem dilengkapi dengan caching untuk 28 variasi konfigurasi, REST API (FastAPI), dan web interface (React + TypeScript)

7.2 Saran Pengembangan

1. Implementasi relevance feedback (Rocchio algorithm)
2. Penambahan ranking algorithm BM25
3. Support untuk dataset lain (Cranfield, Medline)
4. Optimasi performa dengan vectorized computation
5. Docker containerization untuk deployment
6. Unit tests coverage > 80%
7. UI/UX improvement pada frontend (mobile responsive, search history)
8. Implementasi snippet generation untuk search results
9. Support untuk Indonesian language processing (pysaStrawi integration)

Bab 8

Lampiran

8.1 Cara Menjalankan Program

8.1.1 Instalasi Backend

```
1 # Install dependencies
2 uv sync
3
4 # Download NLTK data
5 python -c "import nltk; nltk.download('stopwords'); nltk.download('punkt')"
```

8.1.2 Instalasi Frontend

```
1 cd frontend
2 npm install
```

8.1.3 Menjalankan API Server

```
1 uv run uvicorn api.main:app --reload --host 0.0.0.0 --port 8000
```

8.1.4 Menjalankan Frontend

```
1 cd frontend
2 npm run dev
```

8.1.5 Menjalankan Evaluasi

```
1 uv run python -m src.evaluator
```

8.1.6 Menjalankan dengan Query Expansion

```
1 from src.word2vec_engine import init_engine
2 from src.evaluator import run_experiment
3
4 init_engine() # Load Word2Vec model
5 results = run_experiment(use_expansion=True)
```

8.2 API Endpoints

Method	Endpoint	Deskripsi
GET	/	Health check
POST	/api/search	Search dengan query expansion
POST	/api/expand	Preview expansion terms
GET	/api/index/{doc_id}	Get inverted index
GET	/api/map	Get MAP evaluation report
POST	/api/index/rebuild	Hapus semua index dan cache
POST	/api/map/recalculate/single	Recalculate MAP 1 konfigurasi
POST	/api/map/recalculate/all	Recalculate MAP 28 variasi

Tabel 8.1: API Endpoints

8.3 Referensi

1. Mikolov, T., et al. (2013). "Efficient Estimation of Word Representations in Vector Space." ICLR.
2. Robertson, S.E., et al. (1994). "Okapi at TREC-3." TREC.
3. Manning, C.D., Raghavan, P., & Schütze, H. (2008). "Introduction to Information Retrieval." Cambridge University Press.
4. CISI Dataset: https://ir.dcs.gla.ac.uk/resources/test_collections/cisi
5. Gensim Documentation: <https://radimrehurek.com/gensim/>
6. NLTK Documentation: <https://www.nltk.org/>
7. FastAPI Documentation: <https://fastapi.tiangolo.com/>
8. React Documentation: <https://react.dev/>
9. TailwindCSS Documentation: <https://tailwindcss.com/>